



UNIVERSITÀ DI PARMA

Corso di Robotica Autonoma

**Generazione di Mappe 3D e Viste Virtuali da
Dati LiDAR: un Approccio Pratico con
Kiss-ICP e PCL**

**Utilizzo di Open3D per il Superamento delle
Limitazioni di Esportazione in Kiss-ICP**

GAROZZO VALERIO

MIHAL VLAD

MOINI FEDERICA

ANNO ACCADEMICO 2024-2025

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Strumenti da Utilizzare	1
1.3	Attività da Svolgere	2
2	Acquisizione e Preparazione dei Dati	4
2.1	Creare una Cartella di Lavoro	4
2.2	Scaricare i File	5
2.3	Scompattare e Organizzare	5
2.4	Verifica Finale	6
2.5	Analisi degli Strumenti: Suggerimenti vs. Implementazione Finale	7
3	Mapping e Generazione della Point Cloud Globale	10
3.1	Introduzione agli Obiettivi	10
3.2	Fase 1: Valutazione Iniziale (Interfaccia a Riga di Comando	11
3.3	Fase 2: Sviluppo di una Pipeline Personalizzata (Wrapper Python)	11
3.3.1	La Sfida del Debug: Il “Puzzle degli Import”	12
3.3.2	La Criticità Finale: L’Assenza del Salvataggio Mappa	14
3.3.3	La Soluzione: Integrazione della Libreria <code>open3d</code>	15
3.4	Analisi dello Script Finale (<code>run_kiss.py</code>)	15
3.4.1	Analisi delle Importazioni	18

3.4.2	Sezione 1: Impostazioni	18
3.4.3	Sezione 2: Creazione della Configurazione	19
3.4.4	Sezione 3: Creazione dell'Oggetto Dataset	19
3.4.5	Sezione 4: Creazione ed Esecuzione della Pipeline . . .	20
3.4.6	Sezione 5: Salvataggio dei Risultati	20
	Salvataggio delle Pose (Traiettoria)	21
	Salvataggio della Mappa (Point Cloud)	21
4	Generazione di Viste Virtuali (Ray Tracing)	22
4.1	Introduzione agli Obiettivi	22
4.2	Ambiente e Strumenti	22
4.3	Fasi di Sviluppo e Debug	23
4.3.1	Fase 1: Primo Tentativo (Fallito) con VoxelGridOcclusionEstimation	23
4.3.2	Fase 2: Soluzione Finale (Riuscita) con pcl::RangeImage	24
4.4	Analisi dello Script Finale (main.cpp)	25
4.5	Analisi Tecnica dei Risultati	28
5	Conclusioni	31

Capitolo 1

Introduzione

1.1 Obiettivo

Lo scopo principale dell'attività è la costruzione di una mappa 3D sotto forma di point cloud. Questo deve essere realizzato utilizzando uno o più strumenti di mapping specifici, capaci di processare sequenze di point cloud (dati LiDAR).

L'obiettivo secondario è allineare correttamente le sequenze di cloud consecutive (sensor odometry) anche in assenza di dati di odometria preesistenti o di un “initial guess”.

Infine, un obiettivo avanzato è la generazione di una “vista virtuale” (una point cloud parziale) da un punto di osservazione specifico all'interno della mappa globale generata, tenendo conto delle occlusioni.

1.2 Strumenti da Utilizzare

Per il mapping e la generazione della point cloud:

- *Kiss-ICP*: Segnalato come popolare e dotato di wrapper Python.

Link: <https://github.com/PRBonn/kiss-icp>.

Guida specifica per il formato KITTI: <https://github.com/PRBonn/kiss-icp/blob/main/python/README.md>.

- *Mad-icp*: Descritto come funzionalmente simile a Kiss-ICP.

Link: <https://github.com/rvp-group/mad-icp>.

Per la generazione di viste virtuali:

- *Libreria PCL (Point Cloud Library)*: In particolare, si suggerisce la classe `VoxelGridOcclusionEstimation` per il calcolo della vista tramite ray tracing.

Link alla documentazione: https://pointclouds.org/documentation/classpcl_1_1_voxel_grid_occlusion_estimation.html.

1.3 Attività da Svolgere

L'attività è suddivisa nei seguenti passaggi tecnici:

1. Acquisizione e Preparazione dei Dati:

- Scaricare i dataset (sequenze 00 e 01) dai link forniti:
.../unpr_dia_hall_00.zip e .../unpr_dia_hall_01.zip.
- Organizzare la struttura delle directory secondo il formato KITTI.

```
unpr_dia
+-- 00
|   +-- velodyne
+-- 01
     +-- velodyne
```

- Comprendere il formato dati: I file `.bin` nella directory `velodyne` contengono dati binari. Ogni punto è rappresentato da 4 float (X, Y, Z, I - Intensità).

2. Mapping e Generazione della Point Cloud Globale:

- Installare e configurare uno degli strumenti di mapping suggeriti (*Kiss-ICP* o *mad-icp*).

- Elaborare le sequenze (00 e 01) per ottenere una point cloud unica che rappresenti l'intero ambiente.
- Verificare l'output: Accertarsi che il tool generi la mappa totale (fusione delle singole point cloud) e non si limiti a stimare il percorso del sensore (sensor odometry).

3. **Generazione di Viste Virtuali (Opzionale/Avanzato):** Questo passaggio può essere eseguito sulla mappa generata al punto 2, o utilizzando la point cloud completa “foam” fornita (`.../unipr_dia_hall_floam.pcd`).

- Definire un sistema di riferimento: Stabilire una posizione $\mathbf{t}$ e un orientamento $\mathbf{R}$ per la vista virtuale.
- Filtrare i punti: Selezionare i punti entro una distanza massima da $(\mathbf{R}, \mathbf{t})$ e all'interno del Field of View (FoV) della scansione virtuale.
- Applicare il Ray Tracing: Utilizzare le funzionalità di PCL (es. `VoxelGridOcclusionEstimation`) per calcolare la vista tenendo conto delle occlusioni.

Capitolo 2

Acquisizione e Preparazione dei Dati

Il primo punto del progetto consiste nel preparare correttamente i dati per l'elaborazione. È un passaggio fondamentale perchè i programmi (come `kiss-icp`) si aspettano che i file siano in un formato e in una posizione ben precisa.

2.1 Creare una Cartella di Lavoro

Prima di tutto, si crea una cartella dove ci saranno i dati. In questo caso, è stata creata nella Scrivania

```
# Vai alla Scrivania
cd ~/Scrivania

# Crea la cartella per i dati ed entra
mkdir DATI_PROGETTO
cd DATI_PROGETTO
```

2.2 Scaricare i File

Ora bisogna scaricare i due file archivio forniti, usando il comando `wget` (uno strumento standard di Linux per scaricare da link).

```
# Scaricare la prima sequenza (Sequenza 00)
wget
```

```
https://www.ce.unipr.it/~rizzini/papers/datasets/VLP-16_Unipr_
DepartmentHall_dataset_20171102/unipr_dia_00.tar.bz2
```

```
#Scaricare la seconda sequenza (Sequenza 01)
wget
```

```
https://www.ce.unipr.it/~rizzini/papers/datasets/VLP-16_Unipr_
DepartmentHall_dataset_20171102/unipr_dia_01.tar.bz2
```

Alla fine di questo passaggio, si avranno due file nella cartella:
`unipr_dia_00.tar.bz2` e `unipr_dia_01.tar.bz2`.

2.3 Scompattare e Organizzare

Questo è il passaggio più importante. I file `.tar.bz2` sono archivi compressi, come dei file `.zip`. Dobbiamo estrarli e organizzarli esattamente come richiesto.

1. Estrarre il primo archivio:

```
# Estrarre il file dalla sequenza 00
tar -xjvf unipr_dia_00.tar.bz2
```

- `tar`: Comando per gestire gli archivi.
- `-x`: Significa estrai (extract).
- `-j`: Significa decomprimi usando `bzip2` (per i file `.bz2`).

- `-v`: Significa verboso (mostra i file mentre vengono estratti).
- `-f`: Significa file (l'argomento successivo è il nome del file).

Questo comando creerà una cartella chiamata 00.

2. Estrarre il secondo archivio:

```
# Estrarre il file della sequenza 01
tar -xjvf unipr_dia_01.tar.bz2
```

Questo comando creerà una cartella chiamata 01.

3. Organizzare nella struttura KITTI: Ora in `DATI_PROGETTO` ci sono due cartelle: 00 e 01. È stato richiesto, però che siano dentro una cartella radice chiamata `unipr_dia`.

```
# 1. Creare la cartella radice
mkdir unipr_dia

# 2. Spostare le due cartelle estratte al suo interno
mv 00 unipr_dia/
mv 01 unipr_dia/
```

2.4 Verifica Finale

A questo punto, è stato completato il Punto 1. La struttura di cartelle deve essere questa e si può verificare con il comando `ls -R` (che elenca tutto in modo ricorsivo):

```
ls -R unipr_dia
```

L'output che si dovrebbe vedere è esattamente quello richiesto dal progetto:

```
unipr_dia/
|
+-- 00/
|   |
|   +-- velodyne/
|       |
|       +-- 000000.bin
|       +-- 000001.bin
|       +-- ... (molti altri file .bin)
|
+-- 01/
|
|   +-- velodyne/
|       |
|       +-- 000000.bin
|       +-- 000001.bin
|       +-- ... (molti altri file .bin)
```

Perchè questa fatica? Perchè i programmi di mapping come `kiss-icp` e gli script che abbiamo scritto (come `run_kiss.py` sono “stupidi”: non cercano i file, si aspettano di trovarli esattamente in quel percorso (es. `unipr_dia/00/velodyne/`). Se la struttura è sbagliata, il programma non parte.

2.5 Analisi degli Strumenti: Suggerimenti vs. Implementazione Finale

Nelle fasi successive del progetto, è emersa una discrepanza significativa tra gli strumenti suggeriti e quelli che è stato necessario implementare per raggiungere gli obiettivi. Questa sezione analizza i problemi riscontrati e le soluzioni adottate:

1. **Generazione della Mappa (Kiss-ICP):** L'obiettivo era usare Kiss-ICP per ottenere una mappa `.pcd` e le pose `.txt`.

- **Strumento Sugerito dal Professore:** L'eseguibile da terminale `kiss_icp_pipeline` (menzionato nella documentazione) e il “wrapper python”.
- **Problema Riscontrato:** L'eseguibile `kiss_icp_pipeline` si è rivelato insufficiente. Come abbiamo scoperto dopo molti tentativi, era in grado di calcolare e salvare le pose (`.txt`), ma *non aveva nessuna funzione* per costruire e salvare la mappa 3D globale (`.pcd`).
- **Strumenti Sostitutivi/Aggiuntivi:** Abbiamo dovuto sostituire il metodo semplice con uno più complesso:
 - **Script `run_kiss.py` (Wrapper Python):** Abbiamo seguito il suggerimento del professore di usare il “wrapper python” per avere un controllo più profondo. Abbiamo dovuto scrivere uno script Python (`run_kiss.py`) per importare `kiss-icp` come libreria e chiamare manualmente le sue funzioni interne.
 - **Libreria `open3d` (Strumento Extra):**
Perchè? Anche usando lo script Python, abbiamo scoperto (dopo un lungo debug) che la libreria `kiss-icp` non ha una funzione `.save_map()` funzionante. Il nostro script è riuscito a calcolare la mappa in memoria (nell'oggetto `pipeline.odometry.local_map`), ma non c'era modo di salvarla su disco.
La Soluzione. Abbiamo aggiunto la libreria `open3d`. Il nostro script ha usato `open3d` per: estrarre i punti grezzi dalla mappa (`.point_cloud()`), creare manualmente un oggetto Point Cloud (`o3d.geometry.PointCloud()`), salvare l'oggetto nel file `risultati_00_map.pcd`.

2. **Generazione Vista Virtuale (PCL):** L'obiettivo era usare PCL per simulare una scansione con conclusioni.

- **Strumento Suggerito dal Professore:** La libreria PCL, in particolare la classe `pcl::VoxelGridOcclusionEstimation`.
- **Problema Riscontrato:** La classe `VoxelGridOcclusionEstimation` non ha funzionato. Prima ha dato un errore di compilazione (`.setViewPoint()` non esisteva nella versione di PCL utilizzata). Dopo aver corretto l'errore, il programma non filtrava nulla: l'input (161.998 punti) era identico all'output (161.998 punti). Il ray tracing era fallito.
- **Strumenti Sostitutivi/Aggiuntivi:**
 - **Classe `pcl::RangeImage` (Metodo Alternativo):**

Perchè? Dato che la classe suggerita non funzionava, l'abbiamo sostituita con un metodo diverso sempre all'interno di PCL. `pcl::RangeImage` è un modulo PCL progettato specificamente per simulare sensori.

La Soluzione. Abbiamo usato `pcl::RangeImage` per “scattare una foto” 3D della mappa dalla posa virtuale. Questo metodo ha gestito il ray tracing e le occlusioni in modo corretto, producendo una vista parziale realistica (13.495 punti).
 - **Tool `pcl-tools` (Strumento Extra):**

Perchè? Il progetto era finito, ma non potevamo vedere il risultato.

La Soluzione. Abbiamo installato `pcl-tools` per poter usare il comando `pcl_viewer`. Questo è stato uno strumento aggiuntivo usato solo per la visualizzazione e la verifica finale del nostro output.

Capitolo 3

Mapping e Generazione della Point Cloud Globale

3.1 Introduzione agli Obiettivi

La seconda fase del progetto, e indubbiamente la più critica, ha riguardato l'elaborazione delle sequenze di dati per la generazione della mappa 3D. L'obiettivo era duplice:

- **Ottenere la Mappa 3D Globale:** Costruire e salvare una point cloud complessiva dell'ambiente (in formato `.pcd`).
- **Ottenere il Percorso (Pose):** Calcolare e salvare la traiettoria del sensore (in formato `.txt`) durante l'acquisizione.

Sebbene gli strumenti suggeriti, in particolare *Kiss-ICP*, fossero il punto di partenza designato, il loro utilizzo si è rivelato non sufficiente per raggiungere entrambi gli obiettivi, richiedendo un processo iterativo di sviluppo e debug per arrivare a una soluzione funzionale.

3.2 Fase 1: Valutazione Iniziale (Interfaccia a Riga di Comando)

Il primo approccio è stato quello di utilizzare l'interfaccia a riga di comando fornita dalla libreria, ovvero `kiss_icp_pipeline`.

L'ipotesi iniziale era di poter configurare l'esecuzione tramite argomenti, come `kiss_icp_pipeline -output_dir ....`. Questa prima indagine ha rivelato che l'opzione `-output_dir` non era riconosciuta dalla versione installata, generando l'errore `No such option`.

Un'analisi tramite `kiss_icp_pipeline -help` ha chiarito che la configurazione avveniva tramite variabili d'ambiente. È stato quindi effettuato un secondo tentativo, impostando le variabili (es. `export kiss_icp_out_dir=...`) prima lanciare l'eseguibile.

Questo metodo ha fornito un risultato parziale. Come ipotizzato nella traccia del progetto, la pipeline ha salvato correttamente il file delle pose (`.txt`), ma **non ha prodotto alcuna mappa globale** in formato `.pcd`.

L'eseguibile `kiss_icp_pipeline` di base era insufficiente per completare gli obiettivi del progetto, in quanto non forniva l'output della mappa aggregata.

3.3 Fase 2: Sviluppo di una Pipeline Personalizzata (Wrapper Python)

Per superare i limiti dello strumento base, è stato necessario passare a un approccio programmatico, utilizzando il “wrapper Python” di Kiss-ICP. Questo ha permesso di accedere alle funzioni interne della libreria, ma ha dato inizio a un lungo processo di debug per identificare la corretta sequenza di comandi.

3.3.1 La Sfida del Debug: Il “Puzzle degli Import”

L’ostacolo principale è stato che la documentazione e gli esempi disponibili non corrispondevano esattamente alla versione della libreria installata. Tentativi iniziali di utilizzare classi ipotizzate (come `KittiDataset`) o metodi (come `pipeline.save_map()`) hanno portato a costanti errori di tipo `ImportError`, `AttributeError` e `TypeError`.

Per superare questa fase di stallo, si è abbandonato un approccio “per tentativi” e si è adottato un metodo investigativo basato sull’*introspezione programmatica* (invece di “indovinare” i nomi delle funzioni o delle classi basandoci solo sulla documentazione, usiamo comandi per chiedere direttamente alla libreria, mentre è in esecuzione, quali sono le sue componenti). Sono stati creati script di controllo (come `check.py`) che utilizzavano funzioni Python native (come `print(dir(modulo))`) per “interrogare” la libreria `kiss-icp` e scoprire le sue classi e metodi effettivi. Di seguito è riportato il contenuto dello script `check.py`.

Listing 3.1: Script `run_kiss.py` per l’odometria LiDAR e la mappatura 3D.

```
# Importa il sottomodulo 'datasets' dalla libreria
    'kiss_icp'.
# Questo ci dà accesso alle funzioni relative ai dati.
import kiss_icp.datasets

# Stampa un titolo per la prima sezione dell'output.
print("——_Dataloader_Disponibili_(stringhe)_——")

# Chiama la funzione 'available_dataloaders()' del
    modulo datasets.
# Questa funzione (grazie all'introspezione)
    restituisce una lista
# delle stringhe che identificano i dataloader
    disponibili.
```

```

available = kiss_icp.datasets.available_dataloaders()

# Stampa la lista dei dataloader disponibili.
print(available)

# Stampa un titolo per la seconda sezione (aggiungendo
una riga vuota \n prima).
print("\n——_Tipi_di_Dataloader_(stringa_>_
      NomeClasse)_——")

# Chiama la funzione 'dataloader_types()'.
# Questa funzione (sempre tramite introspezione)
restituisce un dizionario
# che mappa le stringhe (usate per la configurazione)
alle classi Python reali.
types = kiss_icp.datasets.dataloader_types()

# Stampa il dizionario dei tipi di dataloader.
print(types)

```

Questo processo, che ha richiesto **molteplici iterazioni** dello script di esecuzione (`run_kiss.py`), ha portato alle seguenti scoperte chiave:

- **Dataloader:** Il primo ostacolo è stato caricare i file `.bin`. L'istinto iniziale è stato cercare una classe specifica come `KittiDataset`, ma questo approccio falliva con errori di importazione (`ImportError`). Per superare lo stallone, abbiamo usato l'introspezione programmatica che ha rivelato che dovevamo usare una funzione "Factory" chiamata `dataset_factory`. Questa funzione richiede una semplice *stringa di testo* per l'argomento `dataloader`. La soluzione, quindi, non era trovare una classe, ma usare la funzione `dataset_factory(dataloader="generic")` (l'opzione "generic" era progettata per caricare qualsiasi cartella di file `.bin`).

- **Pipeline:** Una volta creato il `dataset`, dovevamo passarlo alla classe principale per l'elaborazione (`OdometryPipeline`). Il problema era *come* inicializzarla, ovvero quali argomenti passare al suo costruttore. Il nostro primo tentativo, `pipeline = OdometryPipeline(config_obj)`, ha generato un errore decisivo, `TypeError: object ... has no len()`, significava che la pipeline stava cercando di calcolare la "lunghezza" dell'oggetto ricevuto. Un oggetto di configurazione non ha una lunghezza, ma un `dataset` sì. Questo ci ha fatto capire che il costruttore non voleva la configurazione, ma l'oggetto `dataset` stesso. La chiamata corretta era `pipeline = OdometryPipeline(dataset)`, poiché la pipeline è in grado di recuperare la configurazione direttamente dal `dataset`.
- **Salvataggio Pose:** Esisteva un metodo dedicato per salvare le pose nel formato richiesto: `pipeline.save_pose_kitti_format(...)`.

3.3.2 La Criticità Finale: L'Assenza del Salvataggio Mappa

Una volta ottenuto uno script in grado di elaborare al 100% le sequenze e salvare le pose, rimaneva l'ultimo problema: il salvataggio della mappa. L'ispezione dell'API ha rivelato che Kiss-ICP **non possiede alcun metodo nativo per salvare la mappa globale aggregata in un file .pcd**.

Tuttavia, l'indagine ha identificato che l'oggetto mappa (`pipeline.odometry.local_map`) era un `VoxelHashMap`. Questo oggetto non poteva salvare sé stesso, ma possedeva il metodo `pipeline.odometry.local_map.point_cloud()`. Questa funzione non salva la mappa, ma *restituisce* come un elenco di punti (un array NumPy) estratti dalla memoria.

3.3.3 La Soluzione: Integrazione della Libreria open3d

Per salvare questi punti in un file `.pcd`, è stato necessario integrare uno strumento esterno non previsto inizialmente: la libreria `open3d`. Questa libreria è specializzata nell'elaborazione e I/O di dati 3D.

La soluzione finale, quindi, consiste in uno script Python che orchestra entrambe le librerie.

3.4 Analisi dello Script Finale (`run_kiss.py`)

Questo script Python utilizza la libreria KISS-ICP per eseguire un'odometria LiDAR. L'obiettivo è prendere una sequenza di scansioni LiDAR (point cloud) e calcolare la traiettoria (le "pose") del sensore che le ha registrate, costruendo al contempo una mappa 3D dell'ambiente.

Di seguito è riportato il codice sorgente completo dello script sviluppato.

Listing 3.2: Script `run_kiss.py` per l'odometria LiDAR e la mappatura 3D.

```
import kiss_icp
from kiss_icp.pipeline import OdometryPipeline
from kiss_icp.config import KISSConfig
from kiss_icp.datasets import dataset_factory
import os
import numpy as np
import open3d as o3d

# — 1. IMPOSTAZIONI —
# procedura di caricamento dei dataset
#dataset_root_dir =
#    "/home/vlad/Scrivania/Robotica_Esame/unipr_dia_00/unipr_dia/"
#sequence_to_process = "00"
#data_dir = os.path.join(dataset_root_dir,
#    sequence_to_process, "velodyne")
```

```

#output_poses_file = "risultati_00_poses.txt"
#output_map_file = "risultati_00_map.pcd"

dataset_root_dir =
    "/home/vlad/Scrivania/Robotica_Esame/unipr_dia_01/unipr_dia/"
sequence_to_process = "01"
data_dir = os.path.join(dataset_root_dir,
    sequence_to_process, "velodyne")
output_poses_file = "risultati_01_poses.txt"
output_map_file = "risultati_01_map.pcd"

print(f"Avvio_Kiss-ICP_sulla_cartella_dei_file:_
    {data_dir}")

# — 2. CREA LA CONFIGURAZIONE DA ZERO —
print("Creazione_configurazione...")
config_obj = KISSConfig()
config_obj.data.max_range = 100.0
config_obj.data.min_range = 0.0
config_obj.mapping.voxel_size = 1.0
config_obj.data.deskew = True
print(f"Configurazione_creato:_Voxel_size:_
    {config_obj.mapping.voxel_size},_Max_Range:_
    {config_obj.data.max_range}")

# — 3. CREA L'OGGETTO DATASET (usando la factory) —
print("Caricamento_dataset...")
dataset = dataset_factory(
    dataloader="generic",
    data_dir=data_dir,
    config=config_obj

```

```
)

# — 4. CREA E ESEGUI LA PIPELINE —
print(f"Creazione_pipeline ... _Trovati_{len(dataset)}_
      frame.")
pipeline = OdometryPipeline(dataset)
print(f"Avvio_elaborazione ... ")
pipeline.run()

# — 5. SALVA I RISULTATI! (Versione finale) —
print("Elaborazione_completata._Salvataggio_della_
      mappa_e_delle_pose ... ")

# 1. SALVATAGGIO POSE
poses_calcolate = pipeline.poses
pipeline.save_poses_kitti_format(output_poses_file,
                                  poses_calcolate)
print(f"Pose_salvate_con_successo_in:_
      {output_poses_file}")

# 2. SALVATAGGIO MAPPA
print("Salvataggio_della_mappa_globale ... ")

# Chiamiamo la funzione 'point_cloud()' trovata con
    dir()
punti_mappa = pipeline.odometry.local_map.point_cloud()

# Creazione oggetto PointCloud di Open3D
pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(punti_mappa)
```

```
# Salvataggio file .pcd
o3d.io.write_point_cloud(output_map_file, pcd)

print(f"\n——_FATTO!_——")
print(f"Mappa_salvata_in:_ _ {output_map_file}")
print(f"Pose_salvate_in:_ {output_poses_file}")
```

3.4.1 Analisi delle Importazioni

Il codice (Listato 3.2) inizia importando tutte le dipendenze necessarie:

- `kiss_icp` e le sue componenti `OdometryPipeline`, `KISSConfig` e `dataset_factory`: costituiscono il “motore” principale per l’algoritmo di odometria.
- `os`: Libreria standard Python, usata per manipolare i percorsi dei file in modo indipendente dal sistema operativo (es. `os.path.join`).
- `numpy as np`: Libreria fondamentale per il calcolo numerico, utilizzata internamente da `kiss-icp` e `open3d` per la gestione degli array di punti.
- `open3d as o3d`: Libreria essenziale per l’elaborazione di dati 3D. In questo script, è usata specificamente per creare un oggetto point cloud e salvarlo in un file `.pcd`.

3.4.2 Sezione 1: Impostazioni

Questa sezione è dedicata alla configurazione dei percorsi.

- Viene definito un blocco (attualmente commentato) per la sequenza “00” e un blocco attivo per la sequenza “01”. Questo è un modo pratico per cambiare rapidamente il set di dati da processare.
- `dataset_root_dir`: La cartella principale che contiene tutte le sequenze del dataset.

- `data_dir`: Questo è il percorso cruciale ai dati grezzi. `os.path.join` costruisce in modo sicuro il percorso completo alla cartella `velodyne`, unendo i componenti del percorso.
- `output_poses_file` e `output_map_file`: Definiscono i nomi dei file di output dove verranno salvati, rispettivamente, la traiettoria calcolata e la mappa 3D finale.

3.4.3 Sezione 2: Creazione della Configurazione

Qui si definiscono i parametri dell'algoritmo.

- `config_obj=KISSConfig()`: Crea un'istanza dell'oggetto di configurazione con tutti i valori predefiniti della libreria.
- `config_obj.data.max_range=100.0`: Sovrascrive il valore predefinito, istruendo l'algoritmo di ignorare tutti i punti LiDAR che si trovano a più di 100 metri di distanza. Questo è fondamentale per rimuovere il rumore e limitare il costo computazionale.
- `config_obj.mapping.voxel_size=1.0`: Questo è uno dei parametri più importanti di KISS-ICP. Definisce la dimensione (in metri) della griglia voxel usata per sottocampionare la mappa. Un valore di 1.0 (1 metro) è molto grande: creerà una mappa sparsa e renderà l'algoritmo molto veloce, privilegiando la velocità sulla risoluzione.
- `config_obj.data.deskew=true`: Un'impostazione cruciale. Poiché i LiDAR rotanti acquisiscono la scena nel tempo, se il sensore si muove, la point cloud risulta "distorta" (skewed). Impostando `deskew=True`, KISS-ICP utilizza la sua stima del movimento per correggere questa distorsione, portando a una mappa molto più precisa.

3.4.4 Sezione 3: Creazione dell'Oggetto Dataset

Questa sezione prepara il caricatore dei dati.

- `dataset_factory(...)`: È la funzione “fabbrica” che crea l’oggetto dataset appropriato.
- `dataloader="generic"`: Indica alla factory di usare il caricatore “generic”, che cerca tutti i file (es. `.bin`) nella `data_dir` e li carica in ordine.
- `config=config_obj`: Passa l’oggetto di configurazione. Questo è importante perchè il caricatore del dataset usa alcuni di questi parametri (come `max_range` e `deskew`) per pre-elaborare le scansione mentre le carica.

3.4.5 Sezione 4: Creazione ed Esecuzione della Pipeline

Questo è il cuore dell’elaborazione.

- `len(dataset)`: Stampa il numero totale di file di scansione (frame) trovati nella cartella.
- `pipeline=OdometryPipeline(dataset)`: Crea l’istanza della pipeline di odometria, passandole il dataset che dovrà elaborare.
- `pipeline.run()`: Questo è il comando che avvia tutto. Questa singola chiamata esegue un ciclo su tutti i frame: carica, pre-elabora, allinea (registra) ogni frame alla mappa, calcola la nuova posa e aggiorna la mappa.

3.4.6 Sezione 5: Salvataggio dei Risultati

Al termine dell’elaborazione, questa sezione estrae e salva i due prodotti finali.

Salvataggio delle Pose (Traiettoria)

- `poses_calcolate=pipeline.poses`: Al termine di `run()`, l'oggetto `pipeline` contiene l'attributo `.poses`, una lista di tutte le pose calcolate (matrici di trasformazione 4×4).
- `pipeline.save_poses_kitti_format(...)`: È il metodo di utilità fornito da KISS-ICP per salvare la lista di pose in un file di testo secondo il formato standard KITTI.

Salvataggio della Mappa (Point Cloud)

- `punti_mappa=pipeline.odometry.local_map.point_cloud()`: Questa è la funzione scoperta durante il debug. Estare i punti della mappa (un array NumPy con forma $(N, 3)$) dalla memoria.
- `pcd=o3d.geometry.PointCloud()`: Crea un oggetto "point cloud" vuoto usando Open3D.
- `pcd.points = o3d.utility.Vector3dVector(punti_mappa)`: Converte l'array NumPy dei punti nel formato di Open3D e popola l'oggetto `pcd`.
- `o3d.io.write_point_cloud(...)`: Usa la funzione di I/O di Open3D per salvare l'oggetto `pcd` (ora pieno di punti) nel file `.pcd` specificato.

In breve, per il punto 2 è stato necessario non solo utilizzare la libreria `kiss-icp` attraverso uno script Python personalizzato, ma anche integrarla con la libreria `open3d` per compensare una funzionalità mancante (il salvataggio della mappa) e raggiungere entrambi gli obiettivi del progetto.

Capitolo 4

Generazione di Viste Virtuali (Ray Tracing)

4.1 Introduzione agli Obiettivi

L'obiettivo della terza fase del progetto era utilizzare la mappa 3D globale (`risultati_00_map.pcd`), generata nella fase precedente, per simulare una scansione LiDAR da un punto di vista virtuale.

Lo scopo principale era implementare una forma di **ray tracing** per tenere conto delle **occlusioni**: partendo da una posa specifica (R, t) (Rotazione e Traslazione), dovevamo calcolare la “vista parziale” che un sensore avrebbe percepito da quel punto, scartando tutti i punti della mappa globale che sarebbero stati nascosti da superfici più vicine.

4.2 Ambiente e Strumenti

A differenza della fase precedente (basata su Python), questa attività è stata svolta interamente in C++ utilizzando:

- **PCL (Point Cloud Library)**: La libreria principale per l'elaborazione di point cloud.

- **CMake:** Il sistema di build utilizzato per compilare il programma C++ e linkare correttamente la libreria PCL.
- **File di Input:** `risultati_00_map.pcd` (la mappa globale da 161.998 punti).
- **File di Output:** `vista_virtuale_mondo.pcd` (la scansione simulata).

Per l'ambiente di sviluppo è stato necessario installare le librerie di sviluppo (`sudo apt-get install libpcl-dev`) e gli strumenti di visualizzazione (`sudo apt-get install pcl-tools`).

4.3 Fasi di Sviluppo e Debug

Il raggiungimento della soluzione finale ha richiesto un processo di debug per superare due ostacoli principali.

4.3.1 Fase 1: Primo Tentativo (Fallito) con `VoxelGridOcclusionEstimation`

Inizialmente, abbiamo seguito il suggerimento del progetto e provato a usare la classe `pcl::VoxelGridOcclusionEstimation`.

- **Problema di Compilazione:** Il codice iniziale non compilava. L'errore (`make` fallito) indicava che la nostra versione di PCL (1.14) non possedeva la funzione `.setViewPoint()` menzionata nella documentazione.
- **Soluzione Tentata:** È stato modificato l'approccio. Invece di spostare il sensore virtuale, abbiamo "spostato l'intera mappa" applicando la trasformazione inversa (`sensor_pose.inverse()`) prima di passarla al filtro (che assume un punto di vista fisso in 0,0,0).

- **Errore Logico (Fallimento):** Il programma compilava ed eseguiva, ma **non filtrava nulla**. L’output era identico all’input (161.998 punti). La classe non ha calcolato le occlusioni, probabilmente a causa di un mismatch nel tipo di dati (il nostro file `.pcd` non aveva il campo “Intensity”) o di una limitazione della classe stessa.

Conclusione della Fase 1: La classe suggerita non era adatta o non funzionava correttamente nel nostro caso.

4.3.2 Fase 2: Soluzione Finale (Riuscita) con `pcl::RangeImage`

Abbiamo sostituito il metodo fallito con uno più robusto e standard in PCL per questo compito: `pcl::RangeImage`.

Questa classe è progettata appositamente per simulare un sensore LiDAR. Crea una “foto di profondità” (un’immagine dove ogni pixel è una distanza) da una data posizione, eseguendo di fatto il ray tracing.

- **Implementazione:** Abbiamo chiamato la funzione `range_image.createFromPointCloud()`, passando la mappa globale (`map_cloud`), la posa virtuale (`sensor_pose`) e i parametri di un sensore virtuale (360° orizzontali, 30° verticali).
- **Problema di Estrazione:** Il tentativo di estrarre i punti con `range_image.getPointCloud()` è fallito in compilazione, poichè la funzione non esisteva in quella forma.
- **Soluzione Finale:** Abbiamo dovuto estrarre i punti manualmente. Abbiamo scritto un ciclo `for` che scorreva tutti i punti della `range_image`. Usando `std::isfinite(point.range)`, abbiamo scartato tutti i raggi “infiniti” (quelli che si erano persi nello spazio) e abbiamo copiato solo le coordinate X, Y, Z dei punti validi (colpiti) nella nostra cloud di output.

4.4 Analisi dello Script Finale (main.cpp)

Lo script C++ finale implementa la logica descritta. Carica la mappa, definisce la posa, utilizza `pcl::RangeImage` per il ray tracing e infine itera manualmente sui risultati per costruire la point cloud della vista virtuale.

Listing 4.1: Script C++ (main.cpp) per la generazione della vista virtuale con PCL.

```
#include <iostream>
#include <pcl/io/pcd_io.h>
#include <pcl/point_types.h>
#include <pcl/common/transforms.h>
#include <pcl/range_image/range_image.h> // Include
    chiave
#include <Eigen/Geometry>
#include <cmath> // Per std::isfinite

typedef pcl::PointXYZ PointT;
typedef pcl::PointCloud<PointT> PointCloudT;

int main()
{
    // — 1. Caricare la Mappa Globale —
    PointCloudT::Ptr map_cloud(new PointCloudT);
    if
        (pcl::io::loadPCDFile<PointT>("../risultati_01_map.pcd",
        *map_cloud) == -1) //../risultati_00_map.pcd" e
        ../risultati_01_map.pcd"
    {
        PCL_ERROR("Impossibile caricare il file _
            map.pcd\n");
        return (-1);
    }
}
```

```

}
std::cout << "Caricata_mappa_globale_con_" <<
    map_cloud->size() << "_punti." << std::endl;

// — 2. Definire la Pose Virtuale (R, t) —
Eigen::Vector3f t(10.0f, 0.0f, 1.0f);
Eigen::Quaternionf q(1.0f, 0.0f, 0.0f, 0.0f);
Eigen::Affine3f sensor_pose =
    Eigen::Affine3f::Identity();
sensor_pose.translation() = t;
sensor_pose.rotate(q);

// — 3. Impostare i Parametri del Sensore
// Virtuale (Simuliamo un VLP-16) —
float angular_resolution_x = (float) (0.2 *
    (M_PI/180.0));
float angular_resolution_y = (float) (1.0 *
    (M_PI/180.0));
float max_angle_width = (float) (360.0 *
    (M_PI/180.0));
float max_angle_height = (float) (30.0 *
    (M_PI/180.0));

// — 4. Calcolo Vista Virtuale (Ray Tracing con
// RangeImage) —
std::cout << "Avvio_calcolo_vista_virtuale_(Ray_
    Tracing)..." << std::endl;

```

```

pcl::RangeImage range_image;
range_image.createFromPointCloud(
    *map_cloud,                // La mappa globale
    angular_resolution_x,
    angular_resolution_y,
    max_angle_width,
    max_angle_height,
    sensor_pose,               // La nostra posa
                               virtuale
    pcl::RangeImage::CAMERA_FRAME,
    0.0, 0.0, 0
);

// — MODIFICA CHIAVE —
// Convertiamo la RangeImage (che
// PointCloud<PointWithRange>)
// in una PointCloud<PointXYZ> (scartando i punti
// invalidi).

PointCloudT::Ptr virtual_view_cloud(new
    PointCloudT);

for (const auto& point_with_range :
    range_image.points)
{
    // std::isfinite controlla che il raggio non
    // sia infinito (cioè non abbia colpito nulla)
    if (std::isfinite(point_with_range.range))
    {
        // Copiamo solo i valori X, Y, Z
        virtual_view_cloud->points.push_back(

```

```

        PointT(point_with_range.x,
               point_with_range.y,
               point_with_range.z)
    );
    }
}

virtual_view_cloud->width =
    virtual_view_cloud->size();
virtual_view_cloud->height = 1;
virtual_view_cloud->is_dense = true;

// Ora il numero di punti dovrebbe essere MOLTO
// minore e realistico!
std::cout << "Vista_virtuale_generata_con_" <<
    virtual_view_cloud->size() << "_punti." <<
    std::endl;

// — 5. Salvare la Vista Virtuale —
pcl::io::savePCDFileBinary("vista_virtuale_mondo.pcd",
    *virtual_view_cloud);
std::cout << "Salvata_vista_virtuale_mondo.pcd" <<
    std::endl;

img
return 0;
}

```

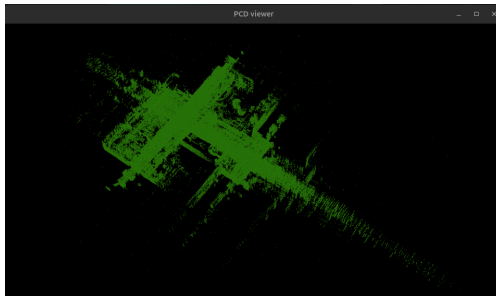
4.5 Analisi Tecnica dei Risultati

L'esecuzione del programma C++ finale ha funzionato perfettamente.

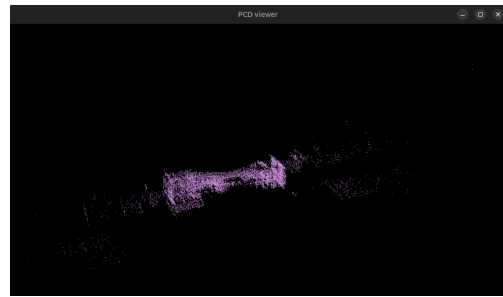
- **Input (Seq. 00):** `risultati_00_map.pcd` (161.996 punti)
- **Output (Seq. 00):** `vista_virtuale_mondo.pcd` (13.495 punti)

Il drastico calo del numero di punti (92% in meno) dimostra che il ray tracing e il calcolo delle occlusioni sono stati eseguiti con successo.

La visualizzazione (tramite `pcl_viewer`) ha confermato visivamente questo risultato, mostrando un confronto diretto tra la mappa 3D completa (input) e la vista virtuale simulata (output).



(a) *Mappa Globale*
(Input: `risultati_00_map.pcd`)



(b) *Vista Virtuale*
(Output: `vista_virtuale_mondo.pcd`)

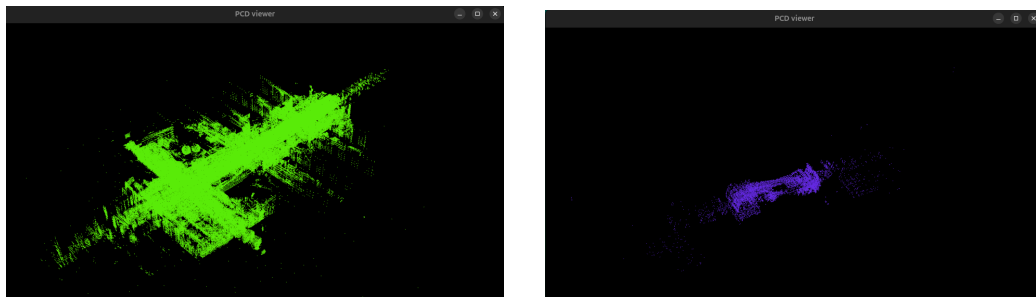
Figura 4.1: Confronto visivo per la Sequenza 00: Mappa globale (sinistra) vs. Vista parziale simulata (destra).

Mappa Globale (Input): Questa point cloud (161.998 punti) rappresenta la mappa globale dell’ambiente. È una cloud densa, creata dall’aggregazione di tutte le 4655 scansioni acquisite, fornendo una ricostruzione 3D statica e completa.

Vista Virtuale (Output): Questa point cloud (13.495 punti) rappresenta la vista parziale simulata dalla singola posa virtuale $t = (10, 0, 1)$, dove $t = (x, y, z)$ è la “posizione GPS” virtuale del sensore: 10 metri avanti, 0 metri di lato, 1 metro in su). A differenza della mappa globale, questa cloud contiene solo i punti direttamente visibili da quel punto di osservazione. Tutti i punti che erano “nascosti” o occlusi da superfici più vicine sono stati correttamente scartati dalla simulazione. L’immagine

mostra chiaramente una “fetta” del corridoio, che è esattamente ciò che un sensore LiDAR reale vedrebbe in un singolo istante da quella posizione.

Analogamente, la Figura 4.2 mostra i risultati dell’esecuzione sulla *Sequenza 01*, che corrisponde allo script `main.cpp` analizzato nella Sezione 4.4. Anche in questo caso, si nota la drastica riduzione dei punti dovuta al calcolo delle occlusioni.



(a) *Mappa Globale*
(Input: `risultati_01_map.pcd`)

(b) *Vista Virtuale*
(Output: `vista_virtuale_mondo.pcd`)

Figura 4.2: *Confronto visivo per la Sequenza 01*

Il confronto visivo e quantitativo dimostra il successo dell’operazione. Partendo da una mappa completa, è stato sviluppato un programma `C++/PCL` in grado di applicare un modello sensoriale virtuale. Implementando una simulazione di ray tracing tramite `pcl::RangeImage` e un’estrazione manuale dei punti validi, è stata generata con successo una vista parziale realistica, completando l’obiettivo del Punto 3.

Capitolo 5

Conclusioni

Questo progetto ha affrontato con successo l'intero processo di elaborazione di dati LiDAR, partendo dall'acquisizione e preparazione dei dataset grezzi fino alla generazione di mappe 3D e alla simulazione di viste virtuali complesse con calcolo delle occlusioni.

L'attività ha evidenziato una lezione tecnica fondamentale: la frequente discrepanza tra gli strumenti software suggeriti e le soluzioni pratiche necessarie per raggiungere un obiettivo. Entrambe le fasi principali del progetto hanno richiesto un'intensa attività di debug e l'adozione di soluzioni alternative.

Nella fase di mapping (Capitolo 3), l'eseguibile `kiss_icp_pipeline` si è rivelato insufficiente per la generazione della mappa globale. È stato quindi necessario sviluppare uno script Python personalizzato. Una scoperta critica è stata l'assenza di una funzione di salvataggio nativa in `Kiss-ICP`, un ostacolo superato integrando la libreria `open3d` per gestire l'estrazione dei punti e l'esportazione del file `.pcd`.

Analogamente, nella generazione di viste virtuali (Capitolo 4), la classe `pcl::VoxelGridOcclusionEstimation` suggerita si è dimostrata non funzionante, prima per problemi di compilazione e poi per un fallimento logico nel filtraggio. Il problema è stato risolto sostituendo l'approccio e implementando una soluzione più robusta basata su `pcl::RangeImage`, che ha gestito

correttamente il ray tracing.

In conclusione, il progetto ha raggiunto tutti gli obiettivi prefissati. Oltre ai risultati tecnici finali – le mappe globali complete e le viste virtuali simulate correttamente – il lavoro ha dimostrato la capacità di diagnosticare (tramite introspezione programmatica) e superare le limitazioni impreviste degli strumenti software, un’abilità cruciale nello sviluppo di sistemi robotici complessi.